TryHackMe – Snapped Phish-ing Line (SOC Level 1)

Investigation Report

Prepared by: Nikhil Gowda | Room type: Challenge | Difficulty: Easy | Platform: TryHackMe (SOC Level 1 path – Phishing module)

---

1. Room Overview

Room name: Snapped Phish-ing Line **Category:** Phishing Analysis / Email Security
Objective: Apply learned skills to probe malicious emails and URLs, exposing a wider phishing campaign.

Scenario: You are IT department personnel at a fictitious company, SwiftSpend Financial. A colleague reports a suspicious email, and you are asked to investigate it — examining the email itself, the authentication mechanisms that should have stopped it, and the malicious infrastructure it points to.

---

2. Objective

- Understand how email-authentication standards (SPF, DKIM, DMARC, S/MIME) are supposed to stop spoofed mail, and check whether the target domain had them configured correctly.
- Analyze a phishing email and the URL it contains.
- Pivot from the phishing URL to the attacker's open web directory and recover the phishing kit.
- Identify what the kit does with harvested credentials (how the stolen data is exfiltrated).
- Locate and decode a hidden flag planted in the attacker's directory structure.

---

3. Tools Used

Tool	Purpose
Browser (Firefox, in-browser VM)	Navigating the malicious domain and its open directory
dmarcian Domain Checker	Validating SPF/DKIM/DMARC records on a domain
Linux terminal (`sha256sum`)	Hashing the downloaded phishing-kit archive for integrity/IOC purposes
Text editor / `cat`	Reading the kit's `log.txt` and PHP source
CyberChef	Decoding the obfuscated flag (Base64 + Reverse)

---

4. Investigation Walkthrough

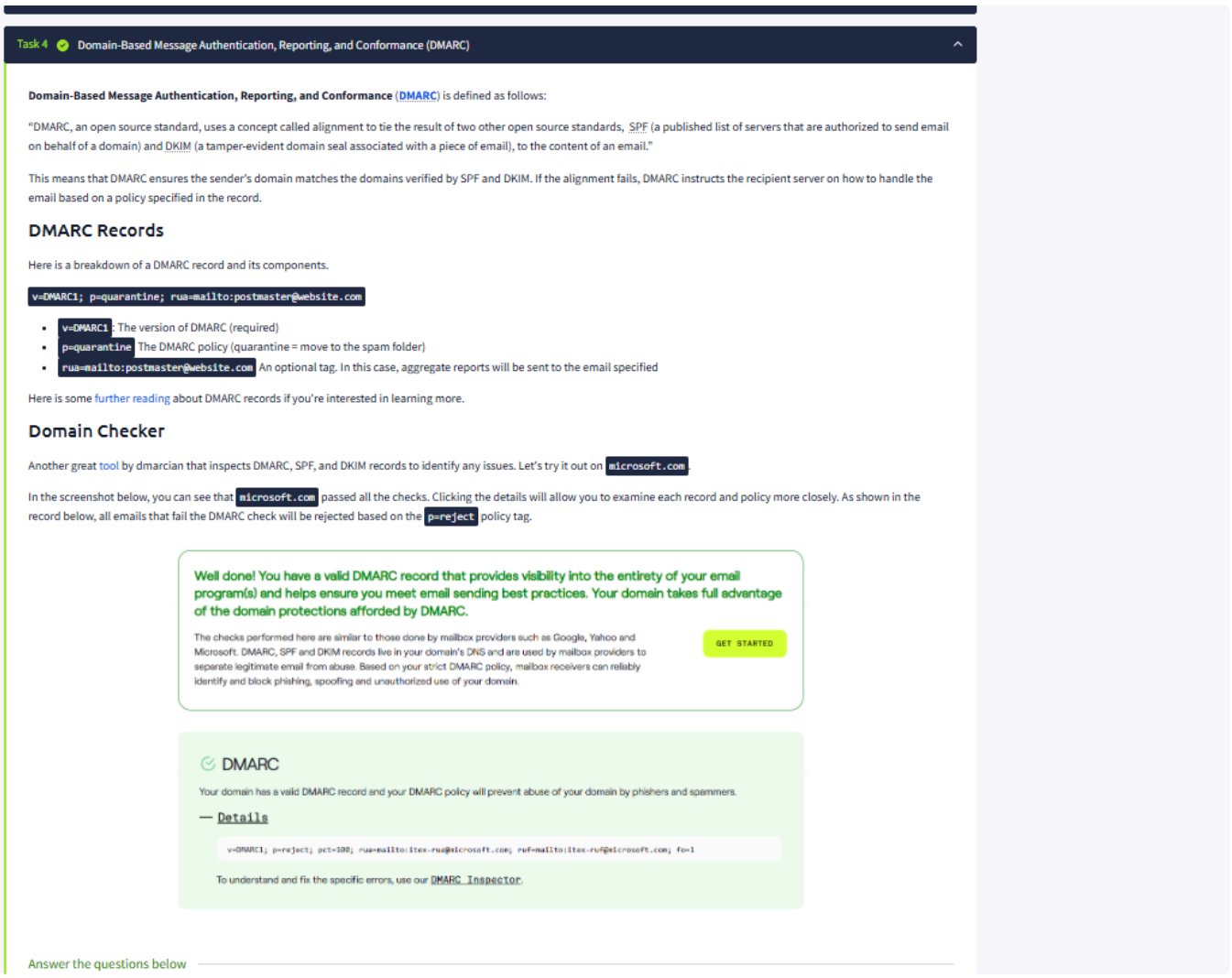
4.1 Email Authentication Concepts Reviewed

Before diving into the malicious URL, the room walks through the email-authentication controls that a well-configured domain should have, since the phishing email's believability depends on how well (or poorly) these are enforced.

DMARC (Domain-Based Message Authentication, Reporting, and Conformance). DMARC ties the results of SPF and DKIM together through a concept called alignment: it checks whether the domain that passed SPF and/or DKIM actually matches the domain shown in the email's "From" header. If alignment fails, the DMARC record tells the receiving mail server what to do with the message, via a policy tag such as `p=quarantine` (send to spam) or `p=reject` (block outright). A typical record looks like:

```
v=DMARC1; p=quarantine; rua=mailto:postmaster@website.com
```

Using dmarcian's public Domain Checker against `microsoft.com` confirmed a valid, strict DMARC record (`p=reject`) in place — meaning any mail that spoofs microsoft.com and fails alignment is rejected outright rather than merely flagged. This was used as the "what good looks like" reference before examining the attacker's domain, which had no such protections.

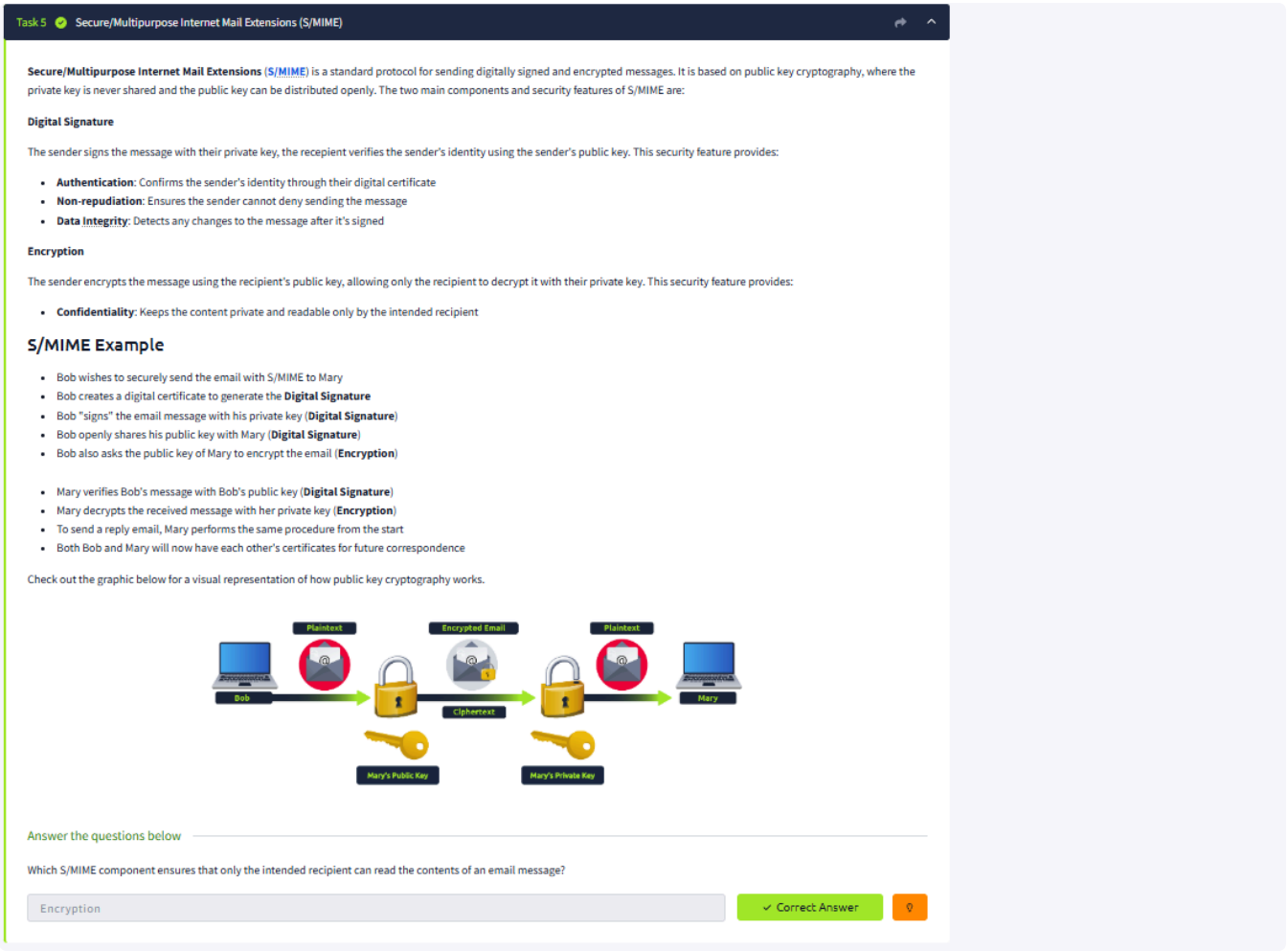


S/MIME (Secure/Multipurpose Internet Mail Extensions). S/MIME is a certificate-based standard for signing and encrypting email using public-key cryptography. Its two components:

- **Digital signature** (signed with the sender's private key, verified with the sender's public key) — provides authentication, non-repudiation, and data integrity.
- **Encryption** (message encrypted with the recipient's public key, decrypted with the recipient's private key) — provides confidentiality, since only the intended recipient can read it.

The room illustrates this with a worked example: Bob signs a message to Mary with his private key and encrypts it with Mary's public key; Mary verifies the signature with Bob's public key and decrypts the message with her own private key. The key takeaway for the investigation: **encryption** is the S/MIME component responsible for keeping an email's

contents unreadable to anyone but the intended recipient — none of which was present in the phishing email under review, which arrived as an unsigned, unencrypted message impersonating an internal Office 365 update notice.



4.2 Analyzing the Phishing Email

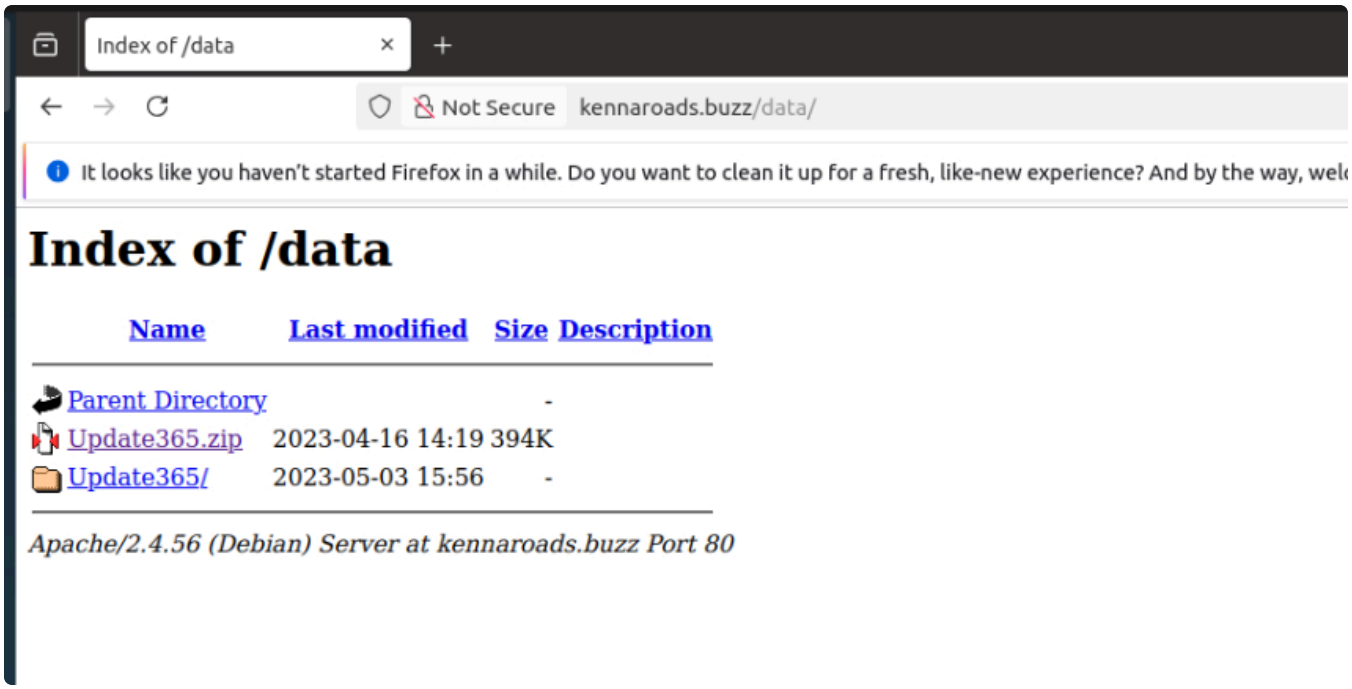
The suspicious email (found under the `phish-emails` directory provided in the attack VM) was disguised as an Office 365 / Outlook update notice. Inspecting the HTML body revealed an embedded link pointing to a lookalike/malicious domain rather than any legitimate Microsoft property:

```
hxxp[://]kennaroads[.]buzz/data/Update365/office365/<hash>/?email=
<victim>@swiftspend.finance&error
```

The link encoded the target's own email address as a URL parameter — a common phishing-kit technique used to pre-fill the fake login form and personalize the lure for each recipient.

4.3 Discovering the Open Directory

Truncating the malicious URL back to its root (`kennaroads.buzz/data/`) exposed a **directory listing** (Apache's default "Index of /" page) that the attacker had left publicly browsable. It listed:



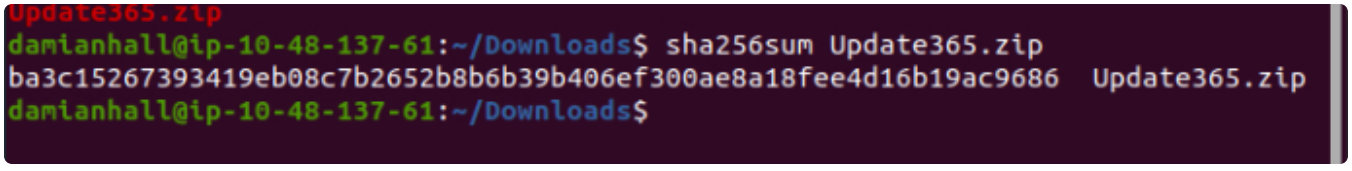
- `Update365.zip` (394 KB, the packaged phishing kit)
- `Update365/` (the same kit, already unpacked)

This kind of misconfiguration — an indexable directory with no access controls — is what let the investigation go from "one phishing link" to "the attacker's entire toolkit."

4.4 Hashing and Inspecting the Kit

`Update365.zip` was downloaded and hashed with `sha256sum` to record it as an indicator of compromise (IOC) before extraction:

```
ba3c15267393419eb08c7b2652b8b6b39b406ef300ae8a18fee4d16b19ac9686
Update365.zip
```



Inside the extracted kit's `Update365/log.txt`, the kit had been silently logging every victim who submitted the fake login form — long before this investigation began. Each entry recorded the client IP, browser user-agent, country, timestamp, and, critically, the **plaintext email address and password** submitted by the victim, labeled with the signature "Created BY Real Carder." Multiple `@swiftspend.finance` employees' credentials appeared in the log (e.g., `michael.ascot@swiftspend.finance`, `zoe.duncan@swiftspend.finance`, `derick.marshall@swiftspend.finance`, `michelle.chen@swiftspend.finance`), confirming the campaign had already succeeded against real users before IT intervened.

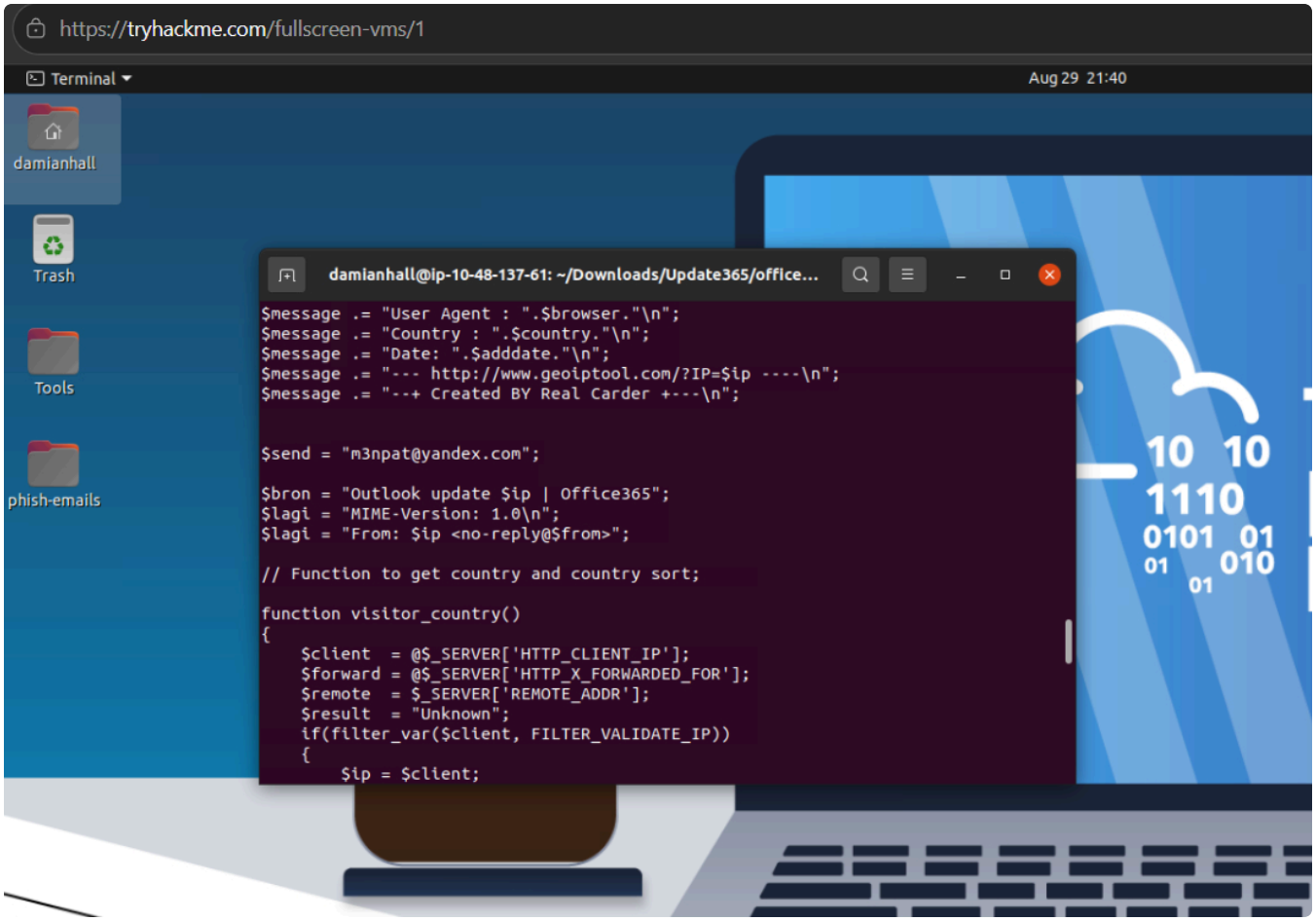


4.5 Reviewing the Kit's Exfiltration Logic

The kit's PHP source (`Update365/office365/...php`) showed exactly how stolen credentials left the victim's browser and reached the attacker. The script built a log message containing the victim's browser user-agent, country, and timestamp, then both:

1. Appended the record to the local `log.txt` (the file reviewed above), and
2. Emailed the same captured details to an attacker-controlled mailbox, `m3npat@yandex.com`, with a deliberately innocuous subject line — `"Outlook update $ip | Office365"` — designed to blend in with legitimate IT notifications if ever noticed in a mail queue or spam trap.

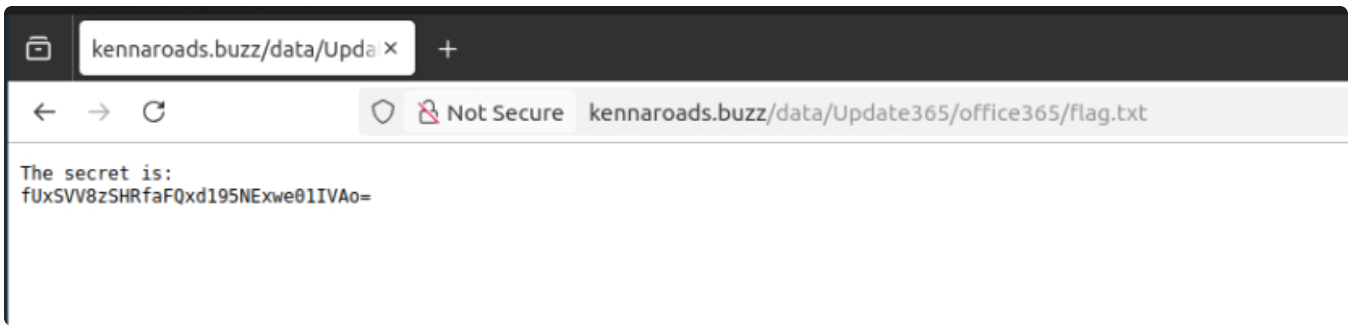
This two-channel approach (local log + email exfiltration) is a common resilience pattern in commodity phishing kits: even if one channel is taken down or blocked, the other still delivers stolen credentials to the operator.



4.6 Finding and Decoding the Hidden Flag

The room's flag was not presented directly; it had to be found by enumerating directories under the kit's path. Continuing to browse the exposed structure down to `Update365/office365/flag.txt` revealed an encoded string:

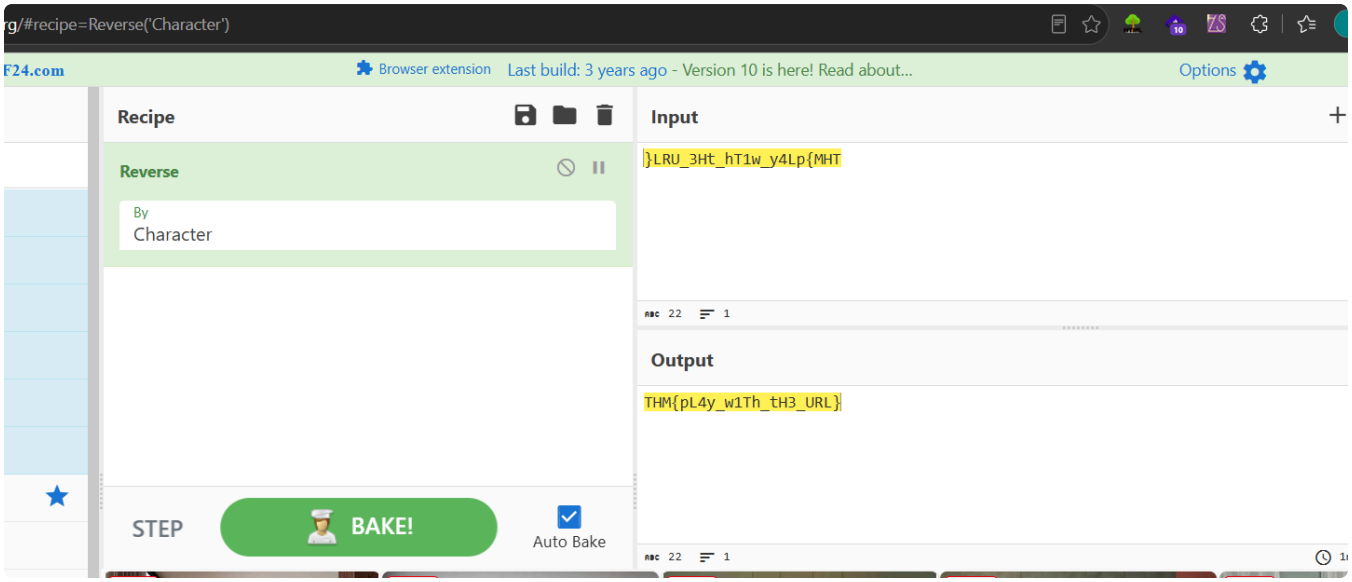
```
The secret is:
fUxSVV8zSHRfaFQxd195NExwe01IVAo=
```



The string is Base64-looking but does not decode cleanly in the normal direction, hinting that the character order itself was reversed before encoding. In CyberChef, applying a **Reverse (by Character)** operation to the string and then a standard **Base64 decode** (or equivalently, decoding Base64 first and reversing the result, depending on how the recipe is chained) recovered the final flag:

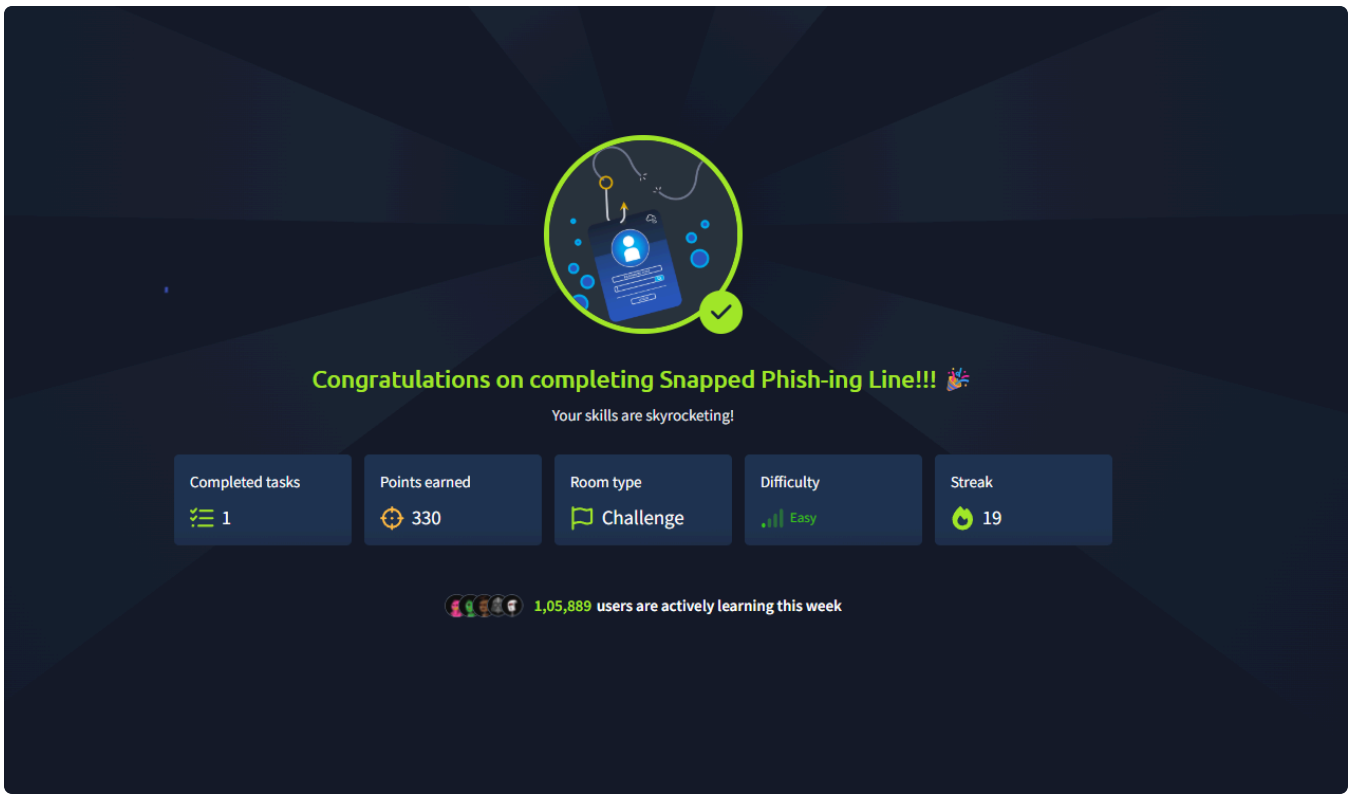
```
THM{pL4y_w1Th_tH3_URL}
```

The flag's own text is a wink at the technique required to solve it — the clue was quite literally to "play with the URL," i.e., manually walk the exposed directory tree rather than relying on a single given link.



4.7 Room Completion

The room was completed as an Easy-difficulty Challenge, earning 330 points for the single consolidated task group.



5. Findings Summary

#	Finding	Category	Severity	Description	Evidence
1	Credential-harvesting phishing kit impersonating Office 365	Phishing / Social Engineering	Critical	A fake Office 365 "update" login page captured victims' email and password in plaintext	`Update365/office365/*.php`, `log.txt`
2	Exposed, indexable web directory on attacker infrastructure	Web Server Misconfiguration	High	`kennaroads.buzz/data/` served a full Apache directory listing, exposing the entire kit and its logs to anyone	Screenshot of "Index of /data"
3	Credentials exfiltrated in plaintext via email	Data Exfiltration	Critical	Harvested email/password pairs were emailed unencrypted to `m3npat@yandex.com` alongside being logged locally	PHP source, `log.txt` entries

#	Finding	Category	Severity	Description	Evidence
4	Victim organization domain lacked sufficient inbound email controls	Email Security Gap	High	The lure reached end users' inboxes and was acted upon by multiple `swiftspend.finance` employees, implying SPF/DKIM/DMARC and/or URL filtering were not effectively blocking or flagging the message	Multiple victim credentials in `log.txt`
5	No S/MIME signing/encryption in use for internal-looking notifications	Email Security Gap	Medium	The spoofed "Outlook update" email had no digital signature a user could check, making it indistinguishable from a real internal notice at a glance	Email sample under `phish-emails`

---

6. Conclusion — How the Attack Was Carried Out

This was a classic **credential-harvesting phishing campaign** built around a commodity phishing kit, executed in the following chain:

1. **Lure delivery:** Victims at SwiftSpend Financial received an email spoofed to look like a routine Office 365/Outlook "update" notification — a pretext chosen because it is mundane enough not to raise suspicion and because it plausibly asks the user to "re-authenticate."
2. **Click-through to a lookalike domain:** The embedded link did not point to any Microsoft property but to `kennaroads.buzz`, a domain registered solely to host the attacker's phishing kit. The victim's own email address was passed in the URL so the fake login page could be pre-populated and look personalized.
3. **Credential capture:** The fake Office 365 login page on `kennaroads.buzz` was a near-identical visual clone of the real Microsoft sign-in flow. Anything a victim typed was captured by server-side PHP rather than forwarded to Microsoft.
4. **Dual-channel exfiltration:** Every captured credential pair was written to a local `log.txt` on the attacker's server **and** simultaneously emailed to the attacker's own mailbox (`m3npat@yandex.com`), disguised with an innocuous subject line. This redundancy meant the operator didn't need to log back into the compromised server to retrieve fresh credentials — they simply checked an inbox.

5. **Operational security failure (on the attacker's side):** The attacker left the hosting directory world-browsable, so the entire kit — including the plaintext log of every victim who had already been compromised — was recoverable by anyone who found or guessed the base URL. This is what allowed the investigation (and this room) to go well beyond the single phishing email and expose the full scope of the campaign.

In short, the attack succeeded because (a) the lure was convincing and low-friction, (b) the organization's inbound email defenses did not stop or sufficiently flag the spoofed message before it reached end users, and (c) at least some employees entered real credentials into the fake page.

---

7. Mitigation Recommendations

Area	Recommendation
Email authentication	Enforce SPF, DKIM, and a strict DMARC policy (`p=quarantine` or `p=reject`) on the organization's sending domain so spoofed "From" addresses are rejected or quarantined before reaching inboxes.
Email gateway / URL filtering	Deploy a secure email gateway with URL rewriting/sandboxing so links are checked (and re-checked) against threat intelligence at click-time, not just at delivery-time.
User awareness training	Run regular phishing-simulation exercises (e.g., fake "update" or "password expiry" lures) so staff learn to verify sender domains and avoid entering credentials on links from email.
Credential hygiene	Enforce multi-factor authentication (MFA) on all Office 365 / email accounts, so a captured password alone is not sufficient for account takeover.
Incident response	As soon as a phishing domain is identified, submit it for takedown/blacklisting (registrar abuse report, browser Safe Browsing report) and force a password reset for any user confirmed in logs like the one recovered here.

Area	Recommendation
Threat hunting	Proactively search outbound/inbound mail logs for the identified indicators (sending domain, attacker mailbox, kit hash) across the wider environment to confirm no other employees were affected.
S/MIME or equivalent	Where feasible, enable S/MIME (or a similar signed-mail standard) for internal system notifications so legitimate IT/Office 365 messages are cryptographically distinguishable from spoofed ones.

---

8. Indicators of Compromise (IOCs)

Type	Value
Malicious domain	`kennaroads.buzz`
Phishing kit archive	`Update365.zip`
Archive SHA-256	`ba3c15267393419eb08c7b2652b8b6b39b406ef300ae8a18fee4d16b19ac9686`
Exfiltration mailbox	`m3npat@yandex.com`
Spoofed theme	Office 365 / Outlook "update" notification
Flag (room completion)	`THM{pL4y_w1Th_tH3_URL}`

---

9. References

- TryHackMe — *Snapped Phish-ing Line* room, SOC Level 1 path: https://tryhackme.com/room/snappedphishingline
- InfoSec Write-ups — *TryHackMe: Snapped Phish-ing Line* walkthrough (community reference used to confirm task flow and flag-finding method)
- dmarcian — Domain Checker tool, used for the DMARC validation step
- CyberChef — used for Base64/Reverse decoding of the room's flag